

MedAssist-AI

Disease Prediction Model Development, Optimization, and System Integration
Sanvi Sawant

CHAPTER 1

PROJECT DESCRIPTION

1.1 Introduction

During Milestone 3 of the MedAssist AI project, my primary responsibility was the development, optimization, and evaluation of the machine learning pipeline for disease prediction. I worked extensively on preparing the medical dataset by performing data preprocessing, exploratory data analysis, identifying class imbalance, implementing downsampling techniques, training multiple machine learning models, and optimizing their performance through hyperparameter tuning. I also evaluated each model using multiple performance metrics to determine the most suitable approach for disease prediction.

In addition to the machine learning pipeline, I was responsible for the complete integration of the application. This involved integrating the React (Vite) frontend, FastAPI backend, authentication module, analytics dashboard, reporting system, and machine learning prediction engine into a single unified application. I configured API communication between all modules, integrated the trained Voting Classifier model with the backend, ensured correct preprocessing and disease mapping during inference, implemented secure session management, and verified role-based access for both patients and doctors.

The objective of my work was to develop a reliable disease prediction pipeline while ensuring seamless communication between every component of the MedAssist AI platform. The final integrated system was validated through end-to-end testing by verifying API endpoints, model loading, dataset integrity, prediction workflows, and overall application stability, resulting in a fully functional healthcare platform ready for deployment and demonstration.

1.2 Objectives

The objectives of my work during Milestone 3 were as follows:

- To perform detailed exploratory data analysis on the medical dataset and understand its characteristics.
- To preprocess the dataset by cleaning the data, encoding disease labels, and preparing it for machine learning.
- To identify and analyze class imbalance within the dataset and implement suitable techniques to improve model performance.
- To train and evaluate multiple machine learning models, for disease prediction.
- To perform hyperparameter tuning to improve the accuracy, robustness, and generalization capability of the trained models.
- To compare model performance using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, Macro F1-Score, and Top-3 Accuracy.
- To contribute to the ensemble learning approach used in the final disease prediction system.
- To integrate the complete application by connecting the React frontend, FastAPI backend, authentication system, analytics module, reporting module, and machine learning pipeline.
- To validate the complete system through end-to-end testing, ensuring reliable communication between all components and successful prediction generation.

CHAPTER 2

DATASET DESCRIPTION

2.1 Overview of the Dataset

During the previous milestones, the machine learning models developed for disease prediction achieved lower-than-expected accuracy. To improve the overall performance of the system, my primary objective during Milestone 3 was to investigate the factors affecting model accuracy and identify suitable optimization techniques. The first step in this process was a detailed analysis of the dataset used for disease prediction.

The dataset consisted of patient symptoms as input features and their corresponding diseases as the target variable, making it a multi-class classification problem. I began by exploring the dataset to understand its structure, quality, and distribution before proceeding with model development. Since machine learning models are highly dependent on the quality and distribution of the training data, this analysis was essential for identifying potential issues that could be affecting prediction performance.

During this analysis, I discovered that the primary challenge was the highly imbalanced distribution of disease classes. While a few diseases contained thousands of samples, many diseases had only a limited number of observations. This imbalance caused the models trained in the earlier milestones to become biased towards majority classes, resulting in lower overall performance and poor prediction of rare diseases. This finding formed the foundation for the preprocessing and optimization techniques implemented throughout Milestone 3.

2.2 Dataset Characteristics

Parameter	Description
Dataset Type	Medical Symptom Dataset
Learning Type	Supervised Learning
Problem Type	Multi-Class Classification
Input Features	Patient Symptoms
Target Variable	Disease

3.1 Dataset Cleaning

The first preprocessing step involved verifying the quality of the dataset. I inspected the dataset for missing values, duplicate records, incorrect data types, and inconsistencies that could negatively affect model training.

The analysis showed that the dataset was generally well-structured and did not contain significant missing values or duplicate records requiring removal. Nevertheless, verifying these aspects was an important step because even a small number of inconsistent records can reduce model reliability and introduce unnecessary bias during training.

I also verified that all symptom features were correctly represented and that the target variable contained valid encoded disease labels before proceeding with further preprocessing.

3.2 Dataset Preparation

After completing the initial preprocessing steps, I prepared the dataset for machine learning by separating the symptom features from the target disease labels. The symptom columns were used as the input features (X), while the encoded disease labels formed the target variable (y). This separation enabled the machine learning models to learn the relationship between different symptom combinations and their corresponding diseases.

To ensure that the model evaluation remained reliable and representative of the overall dataset, I performed a stratified train-test split. Stratification preserved the original distribution of disease classes in both the training and testing datasets, preventing majority classes from dominating either split and ensuring that minority disease classes were also represented during evaluation.

Following the dataset split, I conducted a detailed analysis of the disease distribution within the test dataset. This analysis helped identify rare diseases, defined as diseases having three or fewer samples in the testing set. Since overall accuracy alone does not adequately measure performance on highly imbalanced datasets, I specifically evaluated the model's ability to predict these rare diseases using additional performance metrics such as Macro F1-Score, Top-3 Accuracy, and average precision, recall, and F1-score for rare disease classes. This provided a more comprehensive assessment of the model's performance across both common and infrequent diseases.

3.3 Analysis of Class Imbalance

After preparing the dataset for model training, I conducted a detailed analysis of the disease distribution to understand the reason behind the lower prediction accuracies obtained during the previous milestones. Although the dataset was complete and properly structured, I observed that the number of samples available for each disease varied significantly. A small number of diseases contained thousands of training samples, while many diseases had only a limited number of observations.

This uneven distribution indicated the presence of class imbalance, a common challenge in multi-class classification problems. During model training, machine learning algorithms tend to learn more effectively from classes with a larger number of samples, while classes with very few observations receive comparatively less importance. As a result, the models become

biased toward majority disease classes and struggle to correctly predict rare diseases, even if the overall accuracy appears satisfactory.

To better understand the severity of this imbalance, I analyzed the frequency of each disease class and categorized diseases based on the number of available samples. The analysis confirmed that several diseases contained only a handful of observations, making them difficult for the models to learn effectively. This explained the comparatively lower accuracies achieved during earlier milestones and indicated that improving the dataset representation was more important than simply changing machine learning algorithms.

Since overall accuracy alone could not accurately reflect the model's ability to predict minority disease classes, I decided to include additional evaluation metrics such as Macro F1-Score, Top-3 Accuracy, and separate performance analysis for rare diseases. These metrics provided a more balanced evaluation by considering the performance of both majority and minority classes, enabling a more reliable assessment of model performance.

The findings from this analysis formed the basis for the preprocessing and optimization strategies implemented in the subsequent stages of this milestone, including downsampling experiments, hyperparameter tuning, and ensemble learning.

3.4 Handling Single-Sample Classes

To address the technical limitations of stratified splitting for classes with only one sample, I ensured these specific instances remained within the training set, allowing the model to learn from them without causing split-time errors.

During the preprocessing stage, I observed that a few disease classes contained only a single sample in the entire dataset. Performing a stratified train-test split on such classes is not possible because stratification requires at least one sample to be present in both the training and testing datasets.

To address this issue, I identified all disease classes containing only one sample and ensured that these samples remained exclusively within the training dataset. This approach prevented errors during dataset splitting while still allowing the machine learning models to learn from these rare disease classes during training.

The remaining disease classes, which contained sufficient samples, were then used for stratified train-test splitting. This preprocessing step ensured that the dataset could be divided successfully while preserving the class distribution of all eligible disease categories.

3.5 Stratified Train-Test Split

After handling the single-sample disease classes, I divided the remaining dataset into training and testing sets using a stratified 80:20 train-test split. Stratification ensured that the distribution of disease classes remained approximately consistent across both datasets, allowing the machine learning models to be trained and evaluated on representative data.

The training dataset was used for learning the relationship between symptoms and diseases, while the testing dataset was reserved for evaluating model performance on previously

unseen samples. Maintaining separate datasets helped assess the generalization capability of the trained models and reduced the risk of overfitting.

CHAPTER 4

MODEL DEVELOPMENT AND TRAINING

4.1 Selection of Machine Learning Models

As part of my contribution during Milestone 3, I was responsible for developing and evaluating multiple machine learning models for disease prediction. Initially, four ensemble learning algorithms I selected for experimentation: Random Forest, XGBoost, Gradient Boosting, and LightGBM. These algorithms were chosen because they perform well on structured tabular datasets and are capable of handling multi-class classification problems efficiently.

Each model was trained independently using the preprocessed dataset and subsequently optimized through hyperparameter tuning to improve prediction performance. The objective was to compare the strengths and limitations of each algorithm and identify the models that achieved the best balance between accuracy, robustness, and generalization.

After evaluating the performance of all four models, Random Forest, XGBoost, and LightGBM demonstrated the most suitable performance for ensemble learning and were therefore selected for the final Voting Classifier implemented in the complete MedAssist AI system. Although Gradient Boosting was trained and evaluated as part of my work, it was not included in the final ensemble because the selected combination of models produced better overall performance.

Table 4.1 – Machine Learning Models Developed

Model	Purpose
Random Forest	Baseline ensemble model using bagging
XGBoost	Gradient boosting model with regularization
Gradient Boosting	Performance comparison and ensemble experimentation
LightGBM	Efficient gradient boosting implementation

Voting Classifier (Final)	Final ensemble model using the best-performing algorithms
----------------------------------	---

4.2 Random Forest Model

The first model I developed during Milestone 3 was the Random Forest Classifier. Since Random Forest is an ensemble learning algorithm that combines the predictions of multiple decision trees using the bagging technique, it was selected as the baseline model for the disease prediction task. Its ability to handle high-dimensional tabular datasets and reduce overfitting made it an appropriate starting point for experimentation.

I initially trained the model using its default configuration to establish a baseline performance. After analysing the results, multiple experiments were conducted by modifying important hyperparameters such as the number of decision trees, maximum tree depth, minimum samples required for node splitting, and class weighting strategy. Different combinations of these parameters were evaluated to improve the model's ability to generalize across both majority and minority disease classes.

The optimized Random Forest model demonstrated a significant improvement over the baseline configuration and served as one of the strongest individual models developed during this milestone. Owing to its consistent performance and robustness, it was later selected as one of the constituent models for the final Voting Classifier.

4.3 XGBoost Model

The second model developed during this milestone was Extreme Gradient Boosting (XGBoost). XGBoost is a gradient boosting algorithm that builds decision trees sequentially, where each new tree attempts to correct the prediction errors of the previous trees. Its regularization capabilities and efficient optimization techniques make it one of the most widely used algorithms for structured machine learning datasets.

The model was initially trained using default parameters and subsequently optimized through several experimental iterations. Hyperparameters such as the learning rate, maximum tree depth, number of estimators, and subsampling parameters were adjusted to improve prediction performance while reducing overfitting. The impact of each parameter change was analysed by comparing the resulting evaluation metrics.

Among all the individual models trained during this milestone, XGBoost consistently achieved one of the highest prediction accuracies and demonstrated strong generalization capability. Due to its superior performance, it became one of the primary models selected for the final ensemble learning approach.

4.4 Gradient Boosting Model

The third model implemented as part of my work was the Gradient Boosting Classifier. This algorithm builds an ensemble of weak learners sequentially, where each successive model focuses on correcting the errors made by the previous model. Gradient Boosting was included in the experimentation phase to compare its performance with other ensemble learning algorithms on the disease prediction dataset.

After training the model using the preprocessed dataset, multiple parameter configurations were evaluated to improve its predictive performance. The experiments focused on tuning parameters such as the number of estimators, learning rate, and maximum tree depth. The optimized model was then compared with the remaining algorithms using the selected evaluation metrics.

Although Gradient Boosting produced satisfactory results, its overall performance was comparatively lower than the optimized Random Forest and XGBoost models. Consequently, while it contributed to the comparative analysis, it was not selected for inclusion in the final Voting Classifier.

4.5 LightGBM Model

The final model developed during this milestone was LightGBM, an efficient gradient boosting framework designed for fast training and lower memory consumption. LightGBM was selected because of its ability to process large structured datasets efficiently while maintaining competitive predictive performance.

During experimentation, several preprocessing adjustments were required to ensure compatibility with LightGBM, including feature name standardization. After successfully training the model, multiple hyperparameter combinations involving the number of estimators, maximum tree depth, and minimum child samples were evaluated to improve performance.

Although the individual accuracy of LightGBM was comparatively lower than that of Random Forest and XGBoost, it learned decision boundaries differently and therefore introduced useful diversity during ensemble learning. For this reason, it was retained as one of the constituent models in the final Voting Classifier.

4.6 Hyperparameter Tuning

After training the initial versions of the selected machine learning models, I observed that their default configurations did not provide the best possible performance for the disease prediction task. To improve prediction accuracy and generalization capability, I performed hyperparameter tuning on each model through multiple experimental iterations.

The tuning process involved modifying important model parameters and analysing their impact on the evaluation metrics. Rather than relying on a single configuration, several parameter combinations were tested for each algorithm to determine the most suitable settings for the given dataset. The objective was to improve model performance while minimizing overfitting and ensuring consistent predictions across different disease classes.

The parameters adjusted during experimentation included the number of estimators, maximum tree depth, learning rate, minimum samples required for node splitting, minimum child samples, and other model-specific parameters. Each experiment was evaluated using the predefined performance metrics, and the configuration producing the most balanced results was selected as the final model configuration.

The optimized models obtained through this tuning process were subsequently used for comparative performance analysis and later contributed to the development of the final ensemble Voting Classifier.

Table 4.2 – Hyperparameters Tuned

Model	Parameters Tuned
Random Forest	Number of Estimators, Maximum Depth, Minimum Samples Split, Minimum Samples Leaf, Class Weight
XGBoost	Number of Estimators, Learning Rate, Maximum Depth, Subsample, Column Sample by Tree
LightGBM	Number of Estimators, Maximum Depth, Minimum Child Sample

4.7 Final Model Selection

Following the completion of model training and hyperparameter tuning, I compared the performance of all four machine learning models to determine which algorithms were most suitable for the disease prediction system. The comparison considered multiple evaluation metrics, including accuracy, precision, recall, and F1-score, along with the models' ability to generalize across different disease classes.

Based on the experimental results, Random Forest, XGBoost, and LightGBM demonstrated the most suitable combination of prediction performance and robustness. These three models were therefore selected for the ensemble learning approach adopted in the final system. Although the Gradient Boosting model produced satisfactory results, its performance was comparatively lower than the selected models and was therefore excluded from the final ensemble.

The selected models were later combined using a Voting Classifier, which leveraged the strengths of each individual algorithm to improve overall prediction performance. The detailed performance comparison and evaluation of both the individual models and the final ensemble model are presented in the next chapter.

CHAPTER 5

MODEL EVALUATION AND PERFORMANCE ANALYSIS

5.1 Evaluation Metrics

To evaluate the effectiveness of the trained machine learning models, I used multiple evaluation metrics instead of relying solely on overall accuracy. Since the disease prediction problem involved a large number of disease classes with an imbalanced distribution, a single metric could not completely represent model performance. Therefore, several complementary metrics were selected to assess the models from different perspectives.

Accuracy was used to measure the overall percentage of correctly classified disease instances. Precision measured the correctness of positive predictions, while Recall evaluated the model's ability to identify the actual disease classes. The F1-Score provided a balanced measure by considering both Precision and Recall simultaneously.

In addition to these conventional metrics, Macro F1-Score was used to evaluate the performance of each disease class equally, regardless of the number of available samples. Top-3 Accuracy was also calculated to determine whether the correct disease appeared among the top three predicted diseases. Furthermore, a separate evaluation was performed on rare diseases to analyse the model's capability to predict diseases having very limited testing samples.

Table 5.1 – Evaluation Metrics

Metric	Purpose
Accuracy	Measures the overall percentage of correctly classified instances.
Precision	Measures the correctness of predicted disease classes.
Recall	Measures the ability to identify the actual disease classes.
F1-Score	Balances Precision and Recall.
Macro F1-Score	Gives equal importance to every disease class irrespective of frequency.

LightGBM (Final Downsample d)	66.29	75.18	66.29	71.40	0.464 5	0.8631	Selected for model diversity
--	-------	-------	-------	-------	------------	--------	------------------------------------

5.3 Final Voting Classifier Performance

After evaluating all individual models, the three best-performing algorithms—Random Forest, XGBoost, and LightGBM—were combined using a Voting Classifier to improve the overall prediction performance of the system. The ensemble model leveraged the strengths of each individual classifier, resulting in better robustness and improved generalization compared to standalone models.

The final Voting Classifier achieved the highest overall performance and was therefore selected as the prediction model for deployment within the MedAssist AI application. Its improved accuracy and balanced performance across multiple evaluation metrics demonstrated the effectiveness of ensemble learning for multi-class disease prediction.

Table 5.3 – Final Voting Classifier Performance

Metric	Value
Accuracy	82.95%
Precision	87.92%
Recall	82.95%
F1-Score	84.76%

5.4 Macro F1-Score and Top-3 Accuracy

Since the dataset contained a large number of disease classes with an uneven distribution, additional evaluation metrics were considered to obtain a more comprehensive assessment of the final model.

The Macro F1-Score was used to evaluate the model's performance by assigning equal importance to every disease class, regardless of the number of samples available. This metric ensured that improvements in majority classes did not overshadow the performance on minority diseases.

The Top-3 Accuracy measured whether the correct disease appeared within the model's three highest probability predictions. This metric is particularly useful in healthcare applications, where presenting a shortlist of likely diseases can assist medical professionals during diagnosis.

Table 5.4 – Additional Evaluation Metrics

Metric	Value
Macro F1-Score	0.6034
Top-3 Accuracy	92.69%

5.5 Rare Disease Performance Analysis

To further evaluate the robustness of the final model, I performed a separate analysis on rare diseases, defined as disease classes having three or fewer samples in the testing dataset. Evaluating these classes separately provided insight into the model's ability to generalize beyond frequently occurring diseases.

The results demonstrated that, although predicting rare diseases remains inherently challenging due to the limited amount of training data, the final model was still able to achieve measurable performance across these minority classes. This analysis confirmed that the preprocessing strategies and ensemble learning approach improved the model's capability to handle highly imbalanced disease distributions.

Table 5.5 – Rare Disease Performance

Metric	Value
Precision	0.1975
Recall	0.3092
F1-Score	0.2156

5.6 Summary of Findings

The experimental results showed that hyperparameter tuning and preprocessing significantly improved the performance of the individual machine learning models. Among the standalone models, XGBoost achieved the highest accuracy, while Random Forest provided stable and consistent performance. Although LightGBM individually achieved a lower accuracy, its inclusion in the Voting Classifier introduced diversity that enhanced the overall ensemble performance.

The final Voting Classifier achieved an accuracy of 82.95%, outperforming all individual models and demonstrating the effectiveness of combining multiple classifiers for disease prediction. Additionally, the use of Macro F1-Score, Top-3 Accuracy, and rare disease analysis provided a more comprehensive evaluation of the system, ensuring that performance was assessed across both majority and minority disease classes rather than relying solely on overall accuracy.

CHAPTER 6

SYSTEM INTEGRATION

6.1 Objective of System Integration

After the machine learning pipeline was finalized, my next responsibility was to integrate all the individual project modules into a single fully functional healthcare application. Until this stage, the frontend, backend, authentication system, analytics module, reporting module, and machine learning components had been developed independently by different team members. My objective was to establish seamless communication between these modules and ensure that the complete application functioned as a unified system.

The integration process involved connecting the React (Vite) frontend with the FastAPI backend, configuring API communication, integrating the trained Voting Classifier model, implementing secure authentication, verifying database connectivity, and validating the complete prediction workflow. This stage ensured that every module communicated correctly and produced consistent outputs throughout the application.

6.2 Frontend and Backend Integration

The React (Vite) frontend was integrated with the FastAPI backend by configuring API communication for all major application functionalities. Each frontend component was connected to its respective backend endpoint, enabling real-time communication between the user interface and server-side services.

The integration included APIs for:

- User Authentication
- Symptom Analysis
- Disease Prediction
- Risk Assessment
- Personalized Recommendations
- Report Generation
- Analytics Dashboard

Each API endpoint was tested to ensure correct request handling, response generation, and error management. This integration allowed users to interact with the application through a unified interface while the backend handled all processing and business logic.

6.3 Machine Learning Model Integration

Following the integration of the application modules, I connected the trained Voting Classifier with the backend prediction pipeline. The serialized machine learning model (.pkl) was configured within the FastAPI application to enable real-time disease prediction based on user symptoms.

The integration process included loading the trained model, applying the same preprocessing pipeline used during training, ensuring correct feature ordering, mapping predicted class IDs back to disease names, and generating prediction probabilities. This ensured that the predictions displayed to the user remained consistent with the results obtained during model evaluation.

The complete prediction workflow was validated by testing multiple symptom combinations and verifying that the backend returned accurate disease predictions along with the associated confidence scores.

6.4 Session Management

Session management was enhanced by implementing user-specific prediction caching, ensuring that prediction history and reports remained isolated for each authenticated user. This prevented prediction data from being shared across different user sessions and improved the overall security of the application.

The authentication workflow was thoroughly tested to verify user registration, login, logout, session persistence, and access control for both user roles.

6.5 End-to-End System Testing

After completing the integration of all modules, I performed comprehensive end-to-end testing to verify that the entire application functioned correctly as a single system.

The validation process included:

- Testing all API endpoints.
- Verifying frontend-backend communication.
- Confirming successful model loading.
- Validating prediction generation.
- Testing disease mapping.
- Checking report generation.
- Verifying analytics dashboard functionality.
- Testing patient and doctor workflows.
- Confirming secure authentication.
- Ensuring successful application build and runtime execution.

These tests confirmed that every module communicated correctly with the others and that the complete application operated reliably under normal usage conditions.

6.6 Outcome of Integration

The successful integration of all project components resulted in a fully functional healthcare platform capable of providing disease prediction, symptom analysis, risk assessment, recommendations, reporting, and analytics through a unified web application. By connecting the frontend, backend, authentication system, and machine learning pipeline into a single architecture, the application delivered a seamless user experience while maintaining secure communication and reliable prediction performance.

The completed integration ensured that the trained Voting Classifier could be accessed through the application interface, enabling users to receive disease predictions based on symptom inputs in real time. The final integrated system was successfully validated through end-to-end testing and was ready for demonstration and deployment.

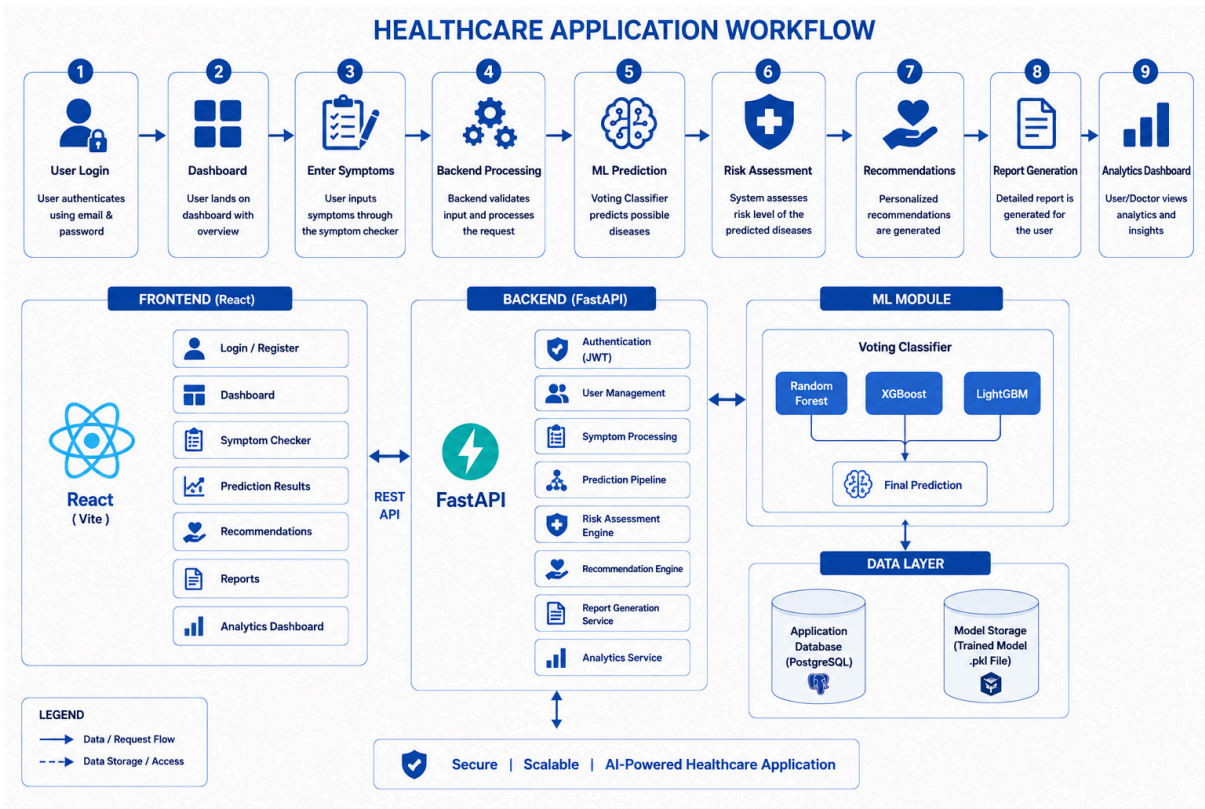
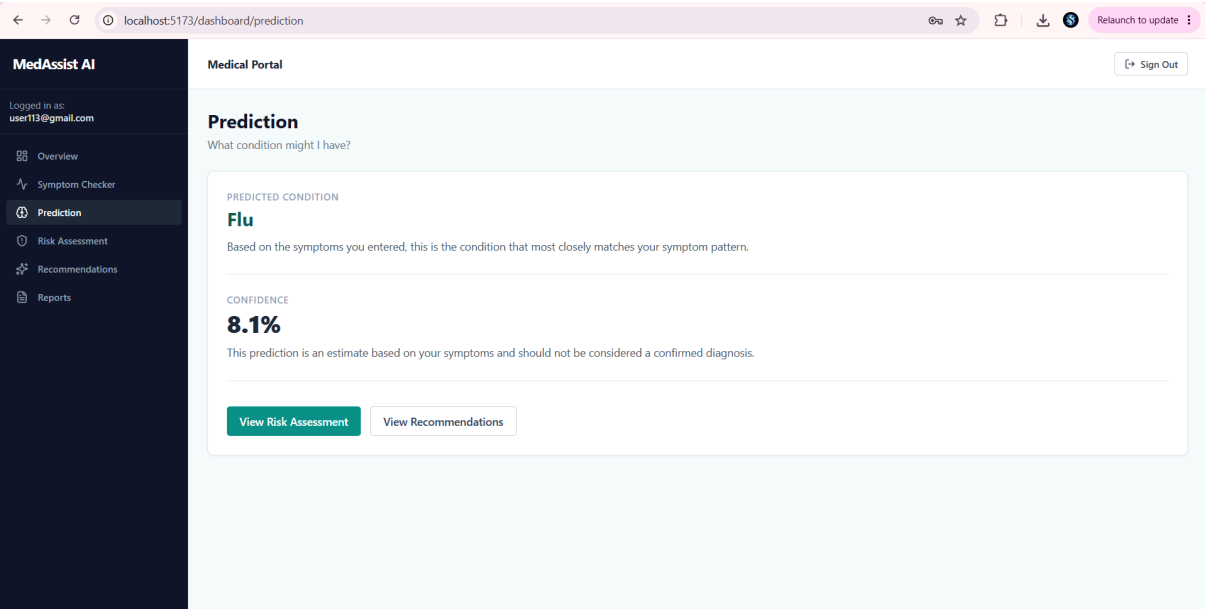
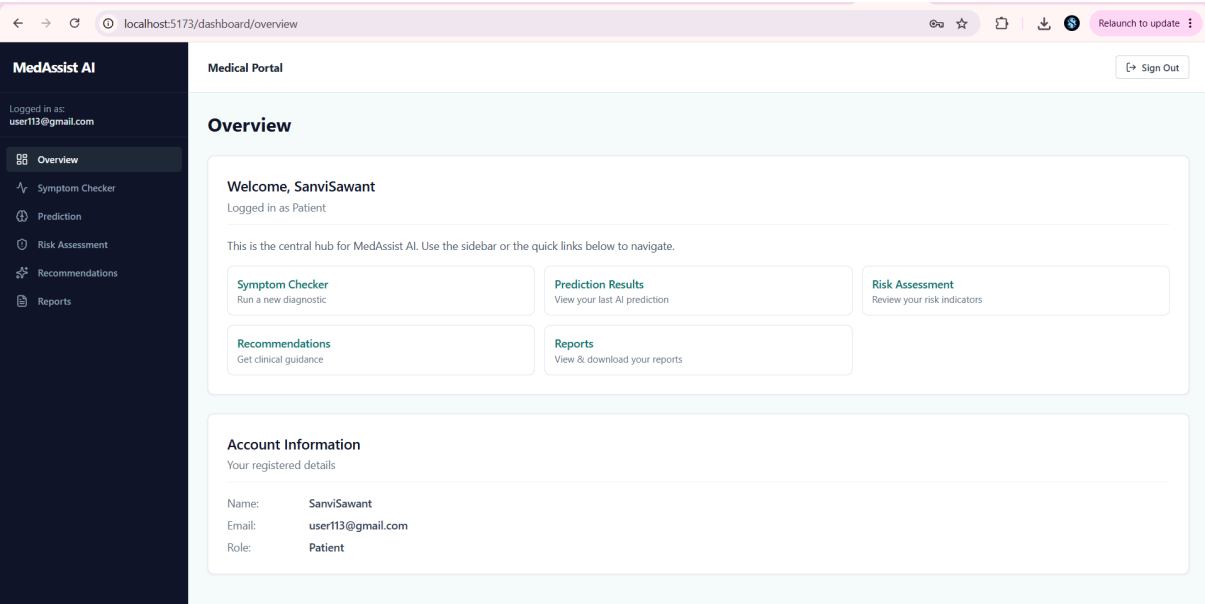


Figure 2 : Medassist AI Workflow

Screenshots :



MedAssist AI

Logged in as:
user113@gmail.com

Overview

Symptom Checker

Prediction

Risk Assessment

Recommendations

Reports

Medical Portal

Sign Out

Risk Assessment

How serious is it?

RISK LEVEL

Medium

Your symptoms suggest that medical attention is recommended, but they do not currently indicate an immediate emergency.

SEVERITY

Moderate (Score: 6/100)

Your symptoms may interfere with daily activities but are generally manageable until you are evaluated by a healthcare professional.

EMERGENCY STATUS

No emergency warning signs were detected from the symptoms provided.

View Recommendations

Back to Prediction

MedAssist AI

Logged in as:
user113@gmail.com

Overview

Symptom Checker

Prediction

Risk Assessment

Recommendations

Reports

Medical Portal

Sign Out

Recommendations

What should I do next?

RECOMMENDED NEXT STEPS

- MODERATE RISK: Schedule a medical consultation soon for proper clinical evaluation.

THINGS TO MONITOR

- Monitor whether symptoms become more frequent or severe.
- Keep note of any new symptoms that develop.

SELF-CARE

- Stay hydrated.
- Get sufficient rest.
- Follow any existing treatment advised by your physician.

SEEK MEDICAL CARE IMMEDIATELY IF

- You experience difficulty breathing.
- Severe chest pain develops.
- Symptoms worsen rapidly.
- You develop any emergency warning signs.

View Risk Assessment

View Prediction

My Reports

MedAssist AI

Logged in as:
user113@gmail.com

Overview

Symptom Checker

Prediction

Risk Assessment

Recommendations

Reports

Medical Portal

Sign Out

Reports

Report Download

Printable report text

Close

=====

MEDASSISTAI MEDICAL HEALTH REPORT

=====

Report Reference ID : #8

Date Generated : 2026-08-07 18:14:51

=====

PATIENT INFORMATION

=====

Patient Name : SanviSawant

Email : user113@gmail.com

Patient ID : #3

=====

AI DISEASE PREDICTION RESULTS

=====

Predicted Disease : Flu

Prediction Confidence: 8.1%

Assessed Risk Level : Medium

=====

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

During Milestone 3, I focused on improving the machine learning pipeline of the MedAssist AI project by identifying the causes of lower prediction accuracy, implementing an effective preprocessing pipeline, training and optimizing multiple machine learning models, and evaluating their performance using comprehensive metrics. Based on the experimental analysis, the best-performing models were incorporated into the final Voting Classifier, which achieved an overall accuracy of 82.95%. In addition to the machine learning work, I successfully integrated the frontend, backend, authentication system, and prediction pipeline into a unified application, ensuring seamless communication between all modules and validating the complete system through end-to-end testing.

7.2 Future Scope

The MedAssist AI system can be further enhanced by incorporating larger and more diverse medical datasets, including additional rare disease samples, to improve model generalization. Future improvements may include the integration of deep learning and Explainable AI (XAI) techniques for more accurate and interpretable predictions, cloud deployment for improved scalability, multilingual support for wider accessibility, and integration with wearable devices and electronic health record (EHR) systems to enable continuous health monitoring and more comprehensive clinical decision support.